Web Development (TWUN13) – Lecture Summary for the Exam

All 9 lectures, summarized in simple English with a little Swedish. Focus = what the 3 old exams actually ask. ★★★ = asked on every exam, ★★ = asked sometimes, ★ = good to know.

How to use this file: read one lecture section, then try the matching old-exam questions. Blue boxes show where a topic showed up on the old exams (E1 = WEB25-19-25, E2 = 2026-01-14, E3 = re-exam 2025-01-09). Orange boxes are traps the exam likes to use. *(Blå rutor = var det kom på tentan. Orange = fällor.)*

Lecture	Topic	Exam weight
L01	HTML & CSS	★★ (1–3 points per exam)
L02	JavaScript basics: types, ==, var/let/const, hoisting, scope	★★★ (4–6 points, often a code question)
L03	DOM, events, event loop, callbacks, Promises, async/await	★★★ (3–6 points)
L04	Intro to React: JSX, components, props, state, keys	★★★ (part of the biggest topic)
L05	React props vs state, useEffect, side effects	★★★ (most code questions, 15+ points)
L06a	React Context (useContext)	★★★ (2–3 points every time)
L06b	React Router	★★ (E3 had 3 questions)
L07	HTTP, Node.js, npm, Express, middleware	★★★ (10–16 points)
L09	Databases (SQL vs NoSQL, MongoDB) + authentication, sessions, cookies	★★★ (3–6 points)
(missing)	Security (STRIDE), Performance (CDN, caching), TypeScript – not in your files	★★★ (4–5 points) – see last section

L01 – HTML & CSS ★★

HTML basics

- **HTML = HyperText Markup Language.** It describes the **structure** of a page. It is NOT a programming language. *(HTML är inget programmeringsspråk, det beskriver struktur.)*
- A page = **elements**. An element = start tag + content + end tag: `<p>Hello</p>`. Tags must be **properly nested** – elements can not overlap. *(Taggar får inte överlappa.)*
- **Attributes** are inside the start tag and refine the tag: `<a href="page.html" target="_blank">`.
- Document structure: `<!DOCTYPE html>` → `<html>` → `<head>` (title, meta, link, style, script) + `<body>` (visible content).
- `<head>` parts: `<title>` = text in the browser tab, `<meta>` = info (charset, description, viewport), `<link>` = load CSS/favicon, `<style>` = local CSS, `<script>` = JavaScript.
- **Viewport meta tag** `<meta name="viewport" content="width=device-width, initial-scale=1.0">` = makes the page fit phones. *(Gör sidan mobilanpassad.)*

Important tags

Tag	What it does
`<a href="url">`	**Hyperlink** ("a" = anchor). `target="_blank"` = new tab, `_self` = same tab (default). `href="#id"` jumps to an element with that id.
`<img src alt width height>`	Image. `alt` = text if the image can't load.
`<ul>`, `<ol>`, `<li>`	Unordered list (bullets), ordered list (numbers), list item.
`<table> <tr> <th> <td> <caption>`	Table, row, header cell, data cell, caption.
`<form action method>` + `<input>`	Form. `action` = where to send, `method` = GET/POST. `<input type="submit">` sends it.
`<b> <strong> <i> <em> <mark> <del> <ins> <sub> <sup>`	Formatting (bold, important, italic, emphasized, highlighted, deleted, inserted, sub/superscript).
` ` `<hr>` `<pre>`	Line break, horizontal line (no end tag for these two), pre-formatted text.
`<!-- comment -->`	HTML comment.
`<article>` `<section>` `<div>`	From the exam: `<article>` = self-contained content (blog post). `<section>` = part of a page. `<div>` = no meaning, just a box.

HTML vs XHTML

- XHTML = HTML 4.01 rewritten with **XML rules**. Stricter: every tag must be closed, all attributes must have values, elements can be nested but not overlap. It is no longer developed.
- So "HTML and XHTML support the same tags and features" is **FALSE**. *(De är inte likadana – XHTML är striktare.)*
- History: HTML5 = W3C Recommendation 2014. Today the **HTML Living Standard** is kept by **WHATWG** (Apple, Google, Mozilla, Microsoft).

CSS basics

- **CSS = Cascading Style Sheets.** A **rule-set** = selector + `{ property: value; }`.
- Selectors: tag `p`, id `#main`, class `.center`, group `h1, h2, p`.
- Three places for CSS: **external** file (`<link rel="stylesheet" href="x.css">`), **internal** (`<style>` in head), **inline** (`style="..."` on the tag).

Cascading order (which rule wins?) ★★

- Rule 1: later rules override earlier ones (top to bottom).
- Rule 2: **id > class > tag > universal (*)**. *(ID väger mest, sen klass, sen tagg.)*
- Rule 3: **inline > internal > external**.
- Rule 4: `!important` beats everything.

> **On the old exams:** E1 Q5: highest specificity among `p.content`, `#main p`, `.content p`, `div > p` → `#main p` (the only one with an id).

Combinators (from the exam, not in the slides)

Selector	Means
`div p`	ALL `<p>` inside a `<div>`, at any depth (descendant). **E2 Q5 answer.**
`div > p`	Only `<p>` that are **direct children** of a `<div>`.
`div + p`	The `<p>` that comes **right after** a `<div>` (sibling).

Box model, units, links

- **Box model** from inside out: **content → padding → border → margin**. Padding is inside the border, margin is outside. *(padding = innanför ramen, margin = utanför.)*
- Shorthand values: 1 value = all sides, 2 = top/bottom + left/right, 3 = top + left/right + bottom, 4 = top, right, bottom, left (clockwise).
- Units: absolute (`px`, `pt`, `cm`) and relative (`em` = element font size, `rem` = root font size, `%` = parent, `vw`/`vh` = 1% of viewport).
- Link states order: `:link`, `:visited`, `:hover`, `:active` (hover after link/visited, active after hover).
- BEM naming (optional): `block__element--modifier`, e.g. `.form__button--highlighted`.

L02 – JavaScript Basics ★★★

This lecture is the base for many "what is logged?" questions. Learn the traps here – they come back every time.

What is JavaScript?

- The only language that runs **natively in the browser**. TypeScript **transpiles** (translates) to JavaScript.
- Three layers of the web: **HTML = structure, CSS = presentation, JavaScript = behaviour.** *(struktur, utseende, beteende.)*
- JS can also run outside the browser with **Node.js**.
- JS vs Java: JS is **dynamically typed** (values have types, variables don't), **prototype-based**, functions are **first-class values**. Errors are often silent.
- **ECMAScript** = the core language. **Web APIs** = extra things the browser gives you (DOM API, Fetch API, Storage API, Geolocation…).
- JS runs in a **sandbox**: it can change the page, make requests, store data, but can NOT read random files or run programs on your computer.
- ES2015 (ES6) brought `let`/`const`, arrow functions, classes, modules, Promises. ES2017 (ES8) brought **async/await**.

var vs let vs const ★★★

	var	let	const
Scope	**Function** scope	**Block** scope `{ }`	**Block** scope `{ }`
Re-assign?	Yes	Yes	**No** (TypeError)
Hoisting	Hoisted **and set to undefined**	Hoisted but not usable → ReferenceError	Same as let
Use	Avoid	Values that change	Default choice

- **const does NOT mean immutable.** `const user = {age: 25}; user.age = 26;` is OK. `user = {}` gives TypeError. Same for arrays: `squares[4] = 25` works. *(const låser variabeln, inte innehållet.)*

Hoisting ★★★ (asked on E1 AND E2)

When JS loads a file, **declarations are moved to the top of their scope** before the code runs. For `var`, the declaration moves up but the **value stays** – so the variable is `undefined` until the assignment line. *(Deklarationen flyttas upp, värdet gör det inte.)*

```
function test() {        //  JS reads it like this:
  console.log(x);        //  var x;            <- undefined
  var x = 5;             //  console.log(x);   -> undefined
}                        //  x = 5;

var x = 5;
function test2() { console.log(x); var x = 10; }
test2();   // undefined  (local var x hides the global x = 5)

console.log(score); let score = 100;  // ReferenceError: Cannot access 'score' before initializatio
```

> **On the old exams:** E1 Q32 (4 points): explain why the output is undefined → **hoisting**. E2 Q8: global `x = 5` + local `var x = 10` → **undefined**.

Scope

- `let`/`const` are **block-scoped**: only visible inside the `{ }` they are declared in. Outside → ReferenceError.
- `var` ignores blocks: `if (true) { var name = "Alice"; } console.log(name)` → "Alice".
- JS uses **lexical scope** = scope is decided by where the code is written, not where it runs.
- Function declarations are hoisted too, so you can call a function before the line where it is written.

Data types

- Primitive: `string`, `number`, `boolean`, `undefined`, `null`, `symbol`, `bigint`. Everything else is an **object** (arrays and functions too).
- `undefined` = value of a variable with no value, a missing object property, or a function with no return. `let x; console.log(x)` → undefined.
- Reading a property that doesn't exist gives `undefined` (no error): `user.hobbies` → undefined.
- Setting a property to `undefined` does NOT delete it. Use `delete user.age`.
- Arrays are objects: you can add properties (`hobbies.user = "Lea"` – length stays 2). `hobbies["1"]` is the same as `hobbies[1]`.
- **Strings are immutable**: `text[0] = "J"` does nothing → still "Hello". (E2 Q6)

Type coercion: == vs === ★★★

- `===` checks **value AND type**. `==` converts types first, then compares value. `5 == "5"` → **true**, `5 === "5"` → false. (E1 Q6)
- `+` with a string = **join as text**: `"5" + 1` → **"51"**. Other math operators convert to number: `"5" - 1` → 4, `"5" * 2` → 10. (E2 Q32, 3 points)
- `4 * "2" ** 3 + 10` → **42** (order of operations: `**` first, "2" becomes number 2).
- **Falsy values:** `false, 0, -0, 0n, "", null, undefined, NaN`. Everything else is **truthy** – even `"0"`, `"false"`, `[]`, `{}`.
- `&&` and `||` do NOT always return true/false. `&&` returns the **first falsy** value, or the **last** value if all are truthy: `"Hello" && 0` → 0, `true && "Hello"` → "Hello". *(&& ger tillbaka ett av värdena, inte alltid true/false.)*
- That is why `show && <h1>Visible</h1>` works in React (E1 Q34). If show is false, React gets `false` and draws nothing.

Functions

- Functions are **first-class values**: store them in variables, pass them to other functions, return them. (E3 Q33: `myFunc(aFuncVar)`)
- Arrow function: `const add = (a, b) => a + b;` – one expression = automatic return.
- Default parameters: `function greet(name = "Guest")`. Rest parameters: `function sum(...numbers)` gives an array.
- Template literals use **backticks** instead of quotes and can include values with ${...} (see code below).
- `fn.toString()` gives the source code of the function as text (E3 Q33).

```
const name = "Alice", age = 25;
console.log(`Hello ${name}, you are ${age} years old.`);   // template literal

function run(fn) { fn(); }          // a function passed as a value
run(() => console.log("Hello!"));
```

Arrays, objects, classes, this

- `forEach` = do something with each item. `map` = make a **new array** with changed items. Also `filter`, `find`, `includes`.
- Objects = key-value pairs. Access with `user.name` or `user["name"]`. A method is a function inside an object.
- Objects inherit from a **prototype** chain: `names → Array.prototype → Object.prototype`.
- Class: `class Person { constructor(name) { this.name = name; } greet() {...} }`, create with `new Person("Alice")`.
- `this`: in a method = the object; alone or in a normal function = the global object; in an HTML event = the element that got the event.

L03 – DOM, Events & Asynchronous JavaScript ★★★

DOM = Document Object Model

- The DOM is a **tree of objects** that represents the HTML page. It is the **API JavaScript uses to read and change the page**. (E2 Q4, E3 Q6 fill-in) *(DOM = trädet som JS använder för att ändra sidan.)*
- Node types: element (1), text (3), comment (8), document (9).
- Select: `getElementById` (one element), `getElementsByTagName` / `getElementsByClassName` (**live HTMLCollection**), `querySelector` (first match), `querySelectorAll` (**static NodeList**).
- NodeList can contain all node types and has `forEach`. HTMLCollection = only elements, no `forEach` (use `Array.from()` first).
- Change: `appendChild`, `insertBefore`, `replaceChild`, `removeChild`, `cloneNode(true)` (deep copy). Create: `document.createElement('p')`, `createTextNode`.
- `textContent` = sets text safely. `innerHTML` = parses HTML – **less secure** (XSS risk), prefer textContent.
- Attributes: `getAttribute`, `setAttribute`, `removeAttribute`. Styles: `node.style.color` (CSS `font-size` → JS `fontSize`). Better: `classList.add/remove/toggle/contains`.
- `<template>` + `template.content.cloneNode(true)` = reusable HTML pieces.

Events

- Use `element.addEventListener('click', handler)` – not `onclick="..."` in HTML (that is called an anti-pattern).
- Ways JS can be added to HTML (E3 Q4): in attributes (onclick), `<script>` in head, `<script>` anywhere → "All of the above".
- Remove: `removeEventListener('click', sameFunction)` – needs a named function.
- `event.preventDefault()` = stop the default action (e.g. a link opening).
- `event.stopPropagation()` = stop the event from going further up/down.
- `event.target` = the element that was actually clicked. `event.currentTarget` = the element the **listener is on**. They are not always the same (E3 Q7).
- **Event delegation:** put ONE listener on a parent instead of one on every child; use `event.target` to see which child was clicked.

Propagation: capture and bubble ★★ (E3 Q34)

An event travels **DOWN** the tree first (capture phase: body → div → button) and then back **UP** (bubble phase: button → div → body). **Bubbling is the default.** A listener with `{ capture: true }` (or `true` as 3rd argument) runs on the way **down**.

```
document.body.addEventListener("click", h, { capture: true });   // runs 1st
container.addEventListener("click", h, { capture: true });       // runs 2nd
button.addEventListener("click", h);                             // runs 3rd (at the target)
// Click on button -> BODY, DIV, BUTTON
```

Event loop ★★★ (asked in some form on every exam)

- **JavaScript is single-threaded** – it does one thing at a time. Events are put in a **queue (FIFO)** and handled **one at a time**, never at the same time. So "events are processed concurrently" is **FALSE** (E3 Q7). *(Ett i taget, aldrig samtidigt.)*
- No code runs unless an event happened, has a handler, and reached the front of the queue.
- Long-running code **blocks** the browser (page freezes). That is why I/O is **asynchronous / non-blocking**.
- **Order of output:** 1) all normal (synchronous) code → 2) **microtasks** (Promise `.then`, code after `await`) → 3) **macrotasks** (`setTimeout`, `setInterval`, I/O callbacks like `fs.readFile`).

```
console.log('start');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('end');
// start -> end -> promise -> timeout     (E1 Q8)

for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);
// 3 3 3   (var = one shared i). With let -> 0 1 2   (E1 Q7)
```

Trap: `setTimeout(fn, 0)` does NOT run right away. It always waits until all normal code is finished (E1 Q3: Start, End, Async Event).

Callbacks

- A **callback** = a function passed into another function, called later. Can be synchronous (`array.sort((a,b) => a-b)`) or asynchronous (`setTimeout`, events, `fs.readFile`).
- Node convention: **error-first callback** `(err, data) => { if (err) ... }`.
- Many nested callbacks = **callback hell** → solved by Promises.

Promises ★★

- A Promise = an object for a value that will come **later**. States: **pending → fulfilled** (resolve) or **rejected** (reject).
- Create: `new Promise((resolve, reject) => { ... })`.
- Consume: `.then(result => ...)`, `.catch(error => ...)`, `.finally(() => ...)` (always runs). They return Promises, so you can **chain** them. Always `return` in a `.then` so the next one gets the value.
- `Promise.all([...])` = run in parallel, fails if ONE fails. `Promise.race([...])` = the first one to finish wins.

async / await ★★★ (E3 Q37)

- `async` before a function → the function **always returns a Promise**.
- `await` (only inside async) **pauses that function** until the Promise is done. The rest of the program keeps running!
- If the Promise is rejected, `await` **throws** – use `try/catch`.

```
async function asyncCall() {
  console.log('calling');
  const result = await resolveAfter2Seconds();   // pauses ONLY this function
  console.log('result: ' + result);
}
const callRet = asyncCall();
console.log('call returns: ' + callRet);
// calling
// call returns: [object Promise]     <- printed right away
// (2 seconds)
// result: resolved
```

L04 – Introduction to React ★★★

Why React and how it thinks

- Build UI from **reusable components**. React keeps the UI in sync with the data (state).
- React is **declarative**: you say WHAT the UI should look like, React does the HOW. You never change the DOM manually. *(Deklarativt = du beskriver vad, inte hur.)*
- **UI = f(state, props)** – the view is a function of the data.

JSX ★★

- **JSX = HTML-like syntax inside JavaScript** (E2 Q9). Browsers don't understand it – it compiles to `React.createElement(type, props, children)`, which creates plain JS objects.
- `<h1 className="title">Hello!</h1>` → `React.createElement("h1", { className: "title" }, "Hello!")`.
- Use `className` (not class) and `htmlFor` (not for). Put JS values inside `{ }`: `<p>Hello {name}</p>`.
- DOM elements are **lowercase** (`div`, `p`). Components are **PascalCase** (`LoginForm`).

Components

- A **functional component** = a normal function that **returns JSX**: `function MyComp() { return <h1>Hello</h1>; }` (E1 Q9).
- `class MyComp extends Component {}` = **class component** (old style). `<MyComp />` just USES a component.
- A component that only shows data from props and has no state = **functional / stateless / presentational component** (E3 Q13).
- Components must be **pure**: same input → same output, no side effects during render.
- `ReactDOM.createRoot(...).render(<App />)` attaches the app to the page.

Hooks

Hook	Use
`useState`	Add **state** to a component (E2 Q10).
`useEffect`	**Side effects**: fetch data, timers, subscriptions (E1 Q10).
`useContext`	Read a value from a **Context**.
`useRef`	Keep a reference (e.g. to a DOM element) without re-rendering.
`useReducer`, `useMemo`, `useCallback`	Complex state / memoize a value / memoize a function.

Props and state

- **Props** = input from **parent → child**. **Read-only** – a child must never change its props. Props is an **object**.
- **State** = data **owned by the component** that can change over time. Created with `const [value, setValue] = useState(initial)`.
- Calling the setter → React **re-renders** the component. Changing a normal variable does NOT re-render.
- **Never change state directly** (`count++` or `user.name = "Anna"`). Always use the setter. For objects use a new object: `setUser({...user, name: "Anna"})`.
- If new state depends on old state, use a function: `setCount(prev => prev + 1)`.
- State updates are **asynchronous and batched** – the value does not change right away (E1 Q30).
- The **initial value** in `useState(0)` is used **only on the first render**. Later renders get the saved value (E2 Q35).
- **Unidirectional data flow**: data goes down (props), actions go up (callbacks). State is often **lifted up** to a common parent to share it.

Lists, keys, fragments, Virtual DOM

- Lists: `items.map(item => <li key={item.id}>{item.name}</li>)`. **Keys** must be unique in that list – they help React see what changed, was added or removed.
- A component must return **one root**. Use a **Fragment** `<> ... </>` (or `<React.Fragment>`) to avoid an extra `<div>`.
- **Virtual DOM**: React builds a new virtual tree, **diffs** it with the old one, and applies only the changes to the real DOM in a batch. Fast.

L05 – React: Props, State & Side Effects (useEffect) ★★★

This is the most important lecture for the exam. The 4 code comprehension questions at the end (15–16 points!) are almost always about useState + useEffect.

When does a component re-render?

- React runs your component function again when (1) it gets **new props** or (2) its **state changes**. *(Ny rendering vid nya props eller ändrat state.)*
- The whole function runs again from top to bottom. That is why code directly in the body runs on EVERY render.

Side effects

- A **side effect** = anything outside calculating the JSX: fetching data, logging, timers, subscriptions, changing the DOM manually, localStorage.
- Side effects belong in **useEffect**, which runs **after** React has updated the screen.
- Lifecycle: **mounting** (added to the page) → **updating** (re-render) → **unmounting** (removed).

useEffect(callback, [dependencies]) ★★★

Dependency array	When the effect runs
none: `useEffect(fn)`	After **every** render (E3 Q12)
empty: `useEffect(fn, [])`	**Only once**, after the first render (E2 Q11)
with values: `useEffect(fn, [a, b])`	After the first render + every time `a` or `b` **changes**

Cleanup

- If the effect **returns a function**, that is the **cleanup**. It runs before the next effect, and when the component is **unmounted**.
- Use it to clear timers (`clearTimeout`), close sockets, remove listeners.

```
function Message() {
  const [visible, setVisible] = useState(true);
  useEffect(() => {
    const id = setTimeout(() => setVisible(false), 2000);  // runs once, after 1st render
    return () => clearTimeout(id);                          // cleanup
  }, []);
  return visible ? <p>Hello!</p> : null;   // after 2 s -> null -> text disappears  (E1 Q35)
}
```

The 4 classic exam patterns ★★★ – learn these by heart

Code	What happens	Why
`setCount(1)` or `setCount(count+1)` **directly in the body**	**Error: "Too many re-renders"** – infinite loop, nothing shown	setState → re-render → setState again → ... (E2 Q12, E2 Q33)
`useEffect(() => setCount(count+1), [count])`	Infinite loop, number counts up forever, React warns	count changes → effect runs again because count is a dependency (E1 Q33)
`useEffect(() => setCount(5), [])` + `console.log(count)`	Logs **0**, then **5**	1st render logs 0, effect runs once → 5 → re-render logs 5 (E2 Q13)
`useEffect(() => console.log(text), [text])` + button sets "Bye"	Logs "Hi", then "Bye" after click	Effect runs after 1st render and when text changes (E1 Q11)

Trap: Fetching data directly in the component body (not in useEffect) gives an infinite loop: render → fetch → setState → render → fetch ... Always fetch inside useEffect.

```
useEffect(() => {
  fetch(`https://swapi.dev/api/people/${count}`)
    .then(res => res.json())
    .then(data => setStarWarsData(data));
}, [count]);   // fetch again only when count changes
```

You might not need an effect

- Don't use an effect to **transform data** for rendering (just compute it in the body) or to **handle user events** (use the event handler).
- Use effects to **sync with external systems** (APIs, databases).

Other useful facts

- Props vs state: props = from above, like function parameters, read-only. State = inside, like local variables, changed with the setter.
- You can't put a custom prop like `blah={true}` on a real DOM element (`<div>`) – real DOM elements only have HTML attributes.
- Spread props: `<Advert {...advert} />` passes all fields of the object as props.
- Fix for `onClick={count++}` (E1 Q13): `onClick={() => setCount(count + 1)}` – give onClick a **function**, and use the **setter**.

L06a – React Context ★★★

- **Problem: prop drilling** = passing props through many components that don't use them, only to reach a deep child. (E2 Q30) *(att skicka props genom många nivåer i onödan.)*
- **Solution: Context** = a "global" value that every component inside the Provider can read directly. Used to **share state between components that are not directly related** (E1 Q12).
- Examples: theme (dark/light), logged-in user, language.

3 steps

```
// 1. Create
const LanguageContext = createContext();

// 2. Provide (wrap the components)
function App() {
  const [language, setLanguage] = useState("english");
  return (
    <LanguageContext.Provider value={{ language, setLanguage }}>   {/* React 19: <LanguageContext value=...>
      <Page />
    </LanguageContext.Provider>
  );
}

// 3. Use (in any child, any depth)
function Button() {
  const { language, setLanguage } = useContext(LanguageContext);
  ...
}
```

- `Context.Provider` **gives the value to the component tree** (E2 Q31). `Context.Consumer` is the old way of reading – now we use `useContext`.
- React finds the **closest Provider above** the component.
- When the Provider's `value` changes, **all components that use that context re-render** (E1 Q31).
- The context does NOT own the state – **useState** in the component with the Provider owns it. To let children change it, put the setter function in the value.
- You can use many contexts in one component (`useContext` several times).
- Don't put everything in Context – it makes components less reusable. Sometimes lifting state up is cleaner.

> **Remember:** Your lecture uses the new React 19 syntax `<UserContext value={user}>`. Old exams use `Context.Provider`. Both mean the same thing – if an answer says **Context.Provider**, pick it.

L06b – React Router ★★

- **Single Page Application (SPA)**: one page, the router swaps components when the URL changes – **no full page reload**. Back/forward buttons and bookmarks still work.
- **React Router** is the library for routing in React (E3 Q8). `npm install react-router-dom`. (Axios = HTTP, Redux = state, Bootstrap = CSS.)
- **Deep linking** = a URL can go **directly to specific content**, e.g. `/products/42`, and can be shared/bookmarked (E3 Q11).
- `<HashRouter>` keeps the location in the **hash (#) part** of the URL: `site.com/#/about` (E3 Q9). `<BrowserRouter>` uses normal paths.

The tools

Tool	What it does
`<BrowserRouter> <Routes> <Route path element>`	**Declarative mode** (simplest).
`createBrowserRouter([...])` + `<RouterProvider>`	**Data mode**: routes as objects, supports **loaders**, **actions**, nested routes, error handling.

Tool	What it does
`<Link to="/x">`	Navigate inside the app without reload. Use `<a>` for external links.
`<NavLink>`	Like Link, but knows if it is **active** (adds class "active").
`useNavigate()`	Navigate from code, e.g. after login: `navigate("/dashboard")`.
`:productId` + `useParams()`	**Dynamic route**: `/products/:productId` → `const { productId } = useParams()`.
`useSearchParams()`	Read/change the **query string** `?sort=date` → `searchParams.get("sort")`.
`<Outlet />`	Where **child routes** are drawn inside the parent. Forget it → children don't show.
`{ index: true }` / `path: "*"`	Default child route / catch-all (404 page).
`loader` + `useLoaderData()`	Load data **before** the route renders (GET-like).
`action` + `<Form method="post">`	**Change** data after a form submit (POST/PUT/DELETE-like).

L07 – HTTP, Node.js & Express ★★★

Web basics

- **Internet** = the network. **The Web** = a service on the Internet that uses **HTTP** (E3 Q1).
- **Client–server model**: the client **requests**, the server **responds**. The server never starts the conversation (E2 Q1).
- **Web app** = software that runs in the browser and talks to a server (E1 Q1). **Static site** = the same fixed files for everyone (E2 Q2). **Dynamic app** = users can change data (bank, Airbnb).
- **HTTP is stateless**: every request is independent and contains all needed info – the server doesn't remember earlier requests (E1 Q2). That's why we need cookies/sessions.
- **Application stack** (e.g. MERN = MongoDB, Express, React, Node) = structured set of tools to develop, deploy and maintain an app (E3 Q2).
- **API** = how programs talk to each other. Web APIs usually use HTTP and send **JSON**. Separate backend (JSON) and frontend (UI) → one backend for web, Android, iOS.

URL ★★

```
https://example.com:3000/api/products/42?sort=date#reviews
protocol  hostname  port  path             query      fragment (hash)
```

- Default ports: **HTTP = 80, HTTPS = 443** (so you usually don't see them).
- "File format" is **NOT** a part of a URL (E3 Q3).

HTTP messages ★★★

Both request and response = **initial line + header lines + empty line + optional body**.

```
GET /api/products/42 HTTP/1.1        <- request line: METHOD  PATH  VERSION
Host: www.example.com                <- headers
Accept: application/json
                                     <- empty line   (these 3 lines = sent by the BROWSER)

HTTP/1.1 200 OK                      <- status line: VERSION  CODE  PHRASE   (from the SERVER)
Content-Type: application/json

{ "id": 42, "name": "Laptop" }       <- body
```

> **On the old exams:** E3 Q21 + Q22: "which lines are sent by the browser?" → the **first three lines** (request line, Host, empty line). Everything else is the server's response. A 404 response is allowed to contain an HTML page.

HTTP methods ★★★

Method	Meaning
GET	**Read** data only.
POST	**Create** / submit data.
PUT	**Replace the whole** resource (E1 Q19).
PATCH	Change **part** of a resource.
DELETE	Delete.
HEAD / OPTIONS	Like GET without body / ask what is allowed.
UPDATE	**Does not exist** (trap answer).

Status codes ★★★ (the lecture says "you should know these")

Group	Codes to know
1xx Informational	100 Continue
2xx Success	**200 OK**, **201 Created**, 202 Accepted, **204 No Content**

Group	Codes to know
3xx Redirection	**301 Moved Permanently**, 302 Found, 304 Not Modified, 307 Temporary Redirect
4xx **Client** error	**400 Bad Request**, **401 Unauthorized** (not logged in), **403 Forbidden** (no permission), **404 Not Found**
5xx **Server** error	**500 Internal Server Error**, 502 Bad Gateway, 503 Service Unavailable

Trap: E3 Q19 writes "304 Moved Permanently" – that is wrong (301 = Moved Permanently, 304 = Not Modified), but it is the only 3xx = redirect option, so pick it.

Headers and body

- Headers = metadata. Know: `Content-Type: application/json`, `Accept: application/json`, `Authorization: Bearer ...`.
- **JSON** = JavaScript Object Notation: numbers, strings, booleans, null, arrays, objects with string keys.

Node.js and npm ★★★

- **Node.js = a runtime environment to run JavaScript on the server** (outside the browser). Not a framework, not a database (E1 Q14). No `window` or DOM.
- Node can do much more than HTTP servers (scripts, tools, files) (E3 Q14). Node modules have **private scope**. Node has an **event queue** like the browser.
- `require('fs')` = **imports a module** into the file (E2 Q15).
- **npm** = package manager. `npm init` → `package.json` (list of dependencies). `npm install express` → saved in `node_modules`. **Never push node_modules to GitHub.**
- `node index.js` runs a file. `nodemon index.js` restarts automatically when files change.
- Node's `EventEmitter` calls all listeners **synchronously, in the order they were registered** (E3 Q16).

Express ★★★

- **Express = a Node.js framework for building web apps and APIs** (E2 Q14).

```
const express = require('express');
const app = express();
app.use(express.json());                        // middleware: parse JSON body into req.body

app.get('/', (req, res) => res.send('Hello World!'));
app.get('/users/:id', (req, res) => {           // dynamic path -> req.params.id
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).send('User not found');
  res.json(user);
});
app.listen(3000, () => console.log('Server ready'));   // START the server  (E2 Q18)
```

Thing	Meaning
`app.METHOD(path, handler)`	**A route = HTTP method + path** (E1 Q15). `app.all()` = all methods.
`req.query`	GET params from URL: `/users?age=25` → `req.query.age`
`req.params`	Path params: `/users/:id` → `req.params.id` (always a **string**)
`req.body`	POST/PUT body. Needs `express.json()` (JSON) or `express.urlencoded()` (forms)
`res.send(x)`	Send a response body
`res.json(obj)`	Send an object as JSON
`res.status(404).send('Not Found')`	Set status **and** send message to the client (E3 Q18)
`res.end()`	Send an empty response
`res.set('Content-Type', 'text/html')`	Set a response header (E3 Q17)

Thing	Meaning
No matching route	Express sends 404 with text **"Cannot GET /path"** (E3 Q36)

Middleware ★★★ (on every exam)

- **Middleware** = a function `(req, res, next)` that runs for requests **before** the route handler. **Registered with** `app.use()` (E2 Q16).
- It can read/change `req` and `res`, **end the cycle** by sending a response (E3 Q15), or call `next()` to go to the **next** middleware (never the previous one).
- Middlewares run **in the order they are registered**, one after another (= **chaining**, E1 Q16).
- It works for **all** HTTP methods, not only GET. It can be async.

```
app.use('/api', (req, res, next) => { console.log('API middleware'); next(); });
app.get('/api', (req, res) => { console.log('API route'); res.send('Done'); });
// GET /api -> "API middleware" then "API route"     (E1 Q17)
```

Input validation

- Never trust user input. Server: `express-validator` (`check('username').isEmail()`), return 422 with errors. Client: `react-hook-form` + `zod`.

L09 – Data Management & Authentication ★★★

Relational databases (SQL)

- Data in **tables** (rows + columns). **Primary key** identifies a row, **foreign key** links tables. **Predefined schema.** Query with **SQL**. Data is **normalized** (split into small parts) and combined with **joins**.
- **ACID**: **A**tomicity (all or nothing), **C**onsistency (always valid state), **I**solation (users don't collide), **D**urability (saved forever after commit).
- Strengths: strong data integrity, powerful joins, ACID. Weaknesses: schema changes are hard, big joins are slow, **horizontal scaling is hard**.

NoSQL ★★★

- NoSQL = **"Not Only SQL"**. Good for huge data, **horizontal scaling**, fast-changing structures, high read/write load.
- **Key advantage: scalability and flexibility** (E1 Q24). Strong ACID by default, fixed schema and complex joins are SQL things (trap answers).
- Flexible / no fixed schema, **no joins** – data that is read together is stored together (**denormalized / embedded**).
- **Vertical scaling** = bigger machine (more CPU/RAM). **Horizontal scaling** = more machines side by side. SQL → vertical, NoSQL → horizontal.
- **Polyglot persistence** = use different databases for different jobs in one app (PostgreSQL for users, Redis for cache, MongoDB for documents).

NoSQL type	Stores	Good for	Example
Key-value	key → value	Cache, sessions, fast lookup	Redis, DynamoDB
Column-family	Row key → column family → columns	IoT/sensor data, logs	Cassandra, Bigtable
Graph	Nodes + relations	Social networks, recommendations	Neo4j
Document	JSON/BSON documents	Flexible data	**MongoDB**, CouchDB

MongoDB ★★★

- **MongoDB = a NoSQL document database** (E1 Q23). Documents are JSON-like (BSON).
- Structure: **database → collection → document**. A **collection holds documents** (like a table) (E2 Q24).
- `_id` = **unique identifier** for each document, created automatically (E2 Q25).
- Schema design in MongoDB = **shape the data after your query patterns** (E1 Q25).
- Queries: `db.users.find()` = `SELECT * FROM users`. `db.users.find({ personID: 3 })` = `WHERE personID = 3`.

Authentication vs authorization

- **Authentication** = proving **who you are** (username + password). *(Vem är du?)*
- **Authorization** = deciding **what you are allowed to do** (admin vs normal user). *(Vad får du göra?)*
- Don't invent your own auth – use established libraries.

Sessions and cookies ★★★

- HTTP is stateless, so we use **sessions** to remember the user across requests.
- **Session-based auth:** login → server checks password → **server creates a session and stores the session data** → sends a **session ID in a cookie** → browser sends the cookie with every request → server finds the session. The **browser only stores the session ID; the real session data is on the server.** (This is why E3 Q29's "session state can be stored client-side" is the wrong statement.)
- **Cookie** = a **small piece of text data** sent by the server and **stored in the browser** (E3 Q28, Q30). Sent **automatically** to the same domain. **Never put sensitive info** (passwords, cards) in

a cookie.
- Server sets cookies (`Set-Cookie`) and reads them; JavaScript can read/change them via `document.cookie` unless `HttpOnly` → "accessed and modified by both client and server" (E2 Q23). Not encrypted by default.
- Cookie attributes: **HttpOnly** (JS can't read it), **Secure** (only over HTTPS), **SameSite** (controls cross-site sending), **Expires/Max-Age** (lifetime).

Passwords and security

- **Never store passwords in plain text. Hash** them (one-way function) with a random **salt** (≥16 bytes). Store salt + hash.
- Salt protects against **rainbow tables**, **brute force**, and gives two users with the same password different hashes.
- Node: `crypto.scrypt(password, salt, 32, cb)` to hash, `crypto.timingSafeEqual()` to compare (prevents timing attacks).
- Common threats: **XSS** (Cross-Site Scripting = evil JavaScript injected into a page), **CSRF** (Cross-Site Request Forgery = the user is tricked into sending a request).

Auth in practice (Passport.js)

- Flow: user enters login → React sends **POST /login** → server finds user, checks hash → wrong: **401** / right: create session + send cookie.
- **Passport** = auth middleware for Express. `LocalStrategy` checks username/password. `express-session` stores the session (default MemoryStore – only for development).
- `serializeUser` = what to save in the session (user.id). `deserializeUser` = load the user → available as `req.user`.
- Protect routes with `req.isAuthenticated()`, e.g. a middleware `isLoggedIn`. Logout: `req.logout()`.
- **CORS with cookies**: server `cors({ origin: 'http://localhost:5173', credentials: true })`, client `fetch(url, { credentials: 'include' })`.

Extra: exam topics that are NOT in your 9 lecture files ★★★

These come up on every exam (about 4–6 points), but there is no lecture file for them. Your numbering jumps from L07 to L09, so **L08 is missing** – try to get it (and any lecture on security/performance) from the course page. Until then, learn this. *(L08 saknas – försök hitta den. Plugga detta så länge.)*

Browser–server communication: fetch & CORS ★★★

- `fetch(url)` = GET by default. `.then(res => res.json())` parses JSON. With options: `fetch(url, { method: 'POST', headers: {...}, body: JSON.stringify(data) })`.
- **BIG TRAP: fetch does NOT reject on 404 or 500.** It only goes to `.catch` on **network errors**. You must check `res.ok` yourself (E1 Q22 vs E2 Q22).
- **CORS** (Cross-Origin Resource Sharing) = lets the browser **access APIs on a different domain**. The server sends CORS headers to allow it. It is not encryption or caching (E1 Q21, E2 Q21 – asked twice!).
- **Postman** = tool to test HTTP APIs (E2 Q20).

```
fetch(url)
  .then(res => { if (!res.ok) throw new Error('Not Found'); return res.json(); })
  .catch(err => console.log(err.message));      // 404 -> "Not Found"   (E1 Q22)

fetch(url).then(res => res.json()).then(d => console.log('Data:', d)).catch(...)
// 500 with JSON body -> logs "Data:" + JSON   (E2 Q22)
```

Security: STRIDE ★★★ (2 points every exam)

Letter	Threat	Simple meaning	Example
S	Spoofing	Pretending to be someone else	Using another user's username/password (E2 Q26)
T	Tampering	Changing data or code	A script changes the homepage
R	Repudiation	Denying you did something, because the system **can't trace/log** actions	E1 Q27
I	Information disclosure	Secret data leaks	Sensitive data over an **unencrypted** channel (E2 Q27), reading files you're not allowed to
D	Denial of Service (DoS)	Making a service **unavailable** to normal users	Flooding the server with traffic (E3 Q27). DDoS = from many machines
E	Elevation of privilege	Getting more rights than you should	Normal user becomes admin

- **SQL injection** = user input becomes SQL code ("Bobby Tables": `Robert'); DROP TABLE Students;--`). Fix: sanitize input / parameterized queries (E1 Q26).
- **HTTPS** encrypts traffic → protects against **man-in-the-middle** attacks. It does NOT protect against SQL injection or DDoS (E3 Q26).

Performance ★★★ (2 points every exam)

Term	Meaning
CDN (Content Delivery Network)	Servers around the world with **cached copies**; the user gets content from the **nearest** one (E1 Q28, E3 Q31).
Caching	**Store temporary data** so you don't fetch/compute it again → faster (E2 Q28).
Load balancing	**Spread incoming traffic over many servers** (E2 Q29).
Lazy loading	Load images/scripts **only when needed** (E1 Q29).
Load testing	Test how the app performs under **expected user load** (E3 Q32).

TypeScript ★

- TypeScript = JavaScript + **types**, transpiles to JS. `any` = **can hold any type** (turns off type checking) (E3 Q10).
- `useState({ price: "100" })` infers price as **string** → `price + 1` = "1001" (string join). Fix: `useState<Product>({ name: "...", price: 100 })` (E3 Q35).

Mongoose and ORM ★

- **Mongoose** = makes MongoDB easier in Node with **schema-based models** (E3 Q23).
- A Mongoose **model** = a **constructor compiled from a Schema**: `const User = mongoose.model('User', userSchema)` (E3 Q24).
- **ORM** (Object-Relational Mapping) = maps objects in your code to database tables/rows (E3 Q25).